

Day 02: Data Types, Variables, Operators & Methods

Java Notes DAY2

1. Data Types

A data type specifies what type of value a variable can store.

Java has two main categories of data types:

A. Primitive Data Types

B. Non-Primitive Data Types

Examples:

String

Arrays

Classes

Interfaces

Objects

Example:

```
String name = "Rahul";
```

Important Points

char stores a single character.

String stores a sequence of characters.

float values normally require f at the end.

long values may require L at the end.

```
float price = 125.50f;
```

```
long population = 1400000000L;
```

2. Variables

A variable is a named memory location used to store a value.

Syntax

```
dataType variableName = value;
```

Example:

```
int age = 25;
double salary = 45000.50;
char grade = 'A';
Declaration
int age;
Initialization
age = 25;
Declaration + Initialization
int age = 25;
Changing a Variable's Value
int age = 20;
```

age = 25;

The value of age becomes 25.

Variable Naming Rules

Valid:

```
int age;
int studentMarks;
int employee_salary;
Invalid:
int 1age;           // Cannot start with a number
int student marks; // Space not allowed
int class;          // Keyword cannot be used
Java is case-sensitive:
int age = 20;
int Age = 30;
age and Age are two different variables.
```

3. Operators

Operators are symbols used to perform operations on values and variables.

A. Arithmetic Operators

Example:

```
int a = 10;
int b = 3;
```

System.out.println(a + b); // 13

System.out.println(a - b); // 7

System.out.println(a * b); // 30

System.out.println(a / b); // 3

System.out.println(a % b); // 1

B. Relational Operators

Relational operators compare two values and return true or false.

Example:

```
int age = 20;
```

System.out.println(age >= 18);

Output:

true

Important

= means assignment.

age = 20;

== means comparison.

age == 20;

C. Logical Operators

Logical operators are mainly used to combine conditions.

AND &&

Returns true when both conditions are true.

age >= 18 && age <= 60

OR ||

Returns true when at least one condition is true.

age < 18 || age > 60

NOT !

Reverses the result.

!true

Result:

false

Example

```
int age = 25;
```

```
if (age >= 18 && age <= 60) {  
    System.out.println("Eligible");  
}
```

D. Increment and Decrement Operators

Increment ++

Increases the value by 1.

```
int count = 5;
count++;
Now:
count = 6
Decrement --
Decreases the value by 1.
int count = 5;
count--;
Now:
count = 4
```

E. Assignment Operators

4. Conditional Statements

Conditional statements are used to make decisions in a program.

For example:

If the marks are 50 or above

Student passes

Otherwise

Student fails

Java provides conditional statements such as:

if

if-else

if-else-if

switch

5. if Statement

The if statement executes a block of code only when the condition is true.

Syntax

```
if (condition) {
// statements
}
```

Example

```
int age = 20;

if (age >= 18) {
System.out.println("Eligible to vote");
}
```

If the condition is false, the statements inside if are not executed.

Flow

Condition

|

+---+---+

true false

||

Execute Skip

6. if-else Statement

if-else is used when there are two possible outcomes.

Syntax

```
if (condition) {  
    // true block  
} else {  
    // false block  
}
```

Example

```
int marks = 45;
```

```
if (marks >= 50) {  
    System.out.println("Pass");  
} else {  
    System.out.println("Fail");  
}
```

If marks >= 50 is true, Pass is displayed.

Otherwise, Fail is displayed.

Real-World Example

```
int age = 17;
```

```
if (age >= 18) {  
    System.out.println("Eligible");  
} else {  
    System.out.println("Not Eligible");  
}
```

7. if-else-if Statement

if-else-if is used when there are multiple conditions.

Syntax

```
if (condition1) {  
    // statements  
}  
else if (condition2) {  
    // statements  
}  
else if (condition3) {  
    // statements  
}  
else {  
    // statements  
}
```

Example: Student Grade

```
int marks = 82;  
  
if (marks >= 90) {  
    System.out.println("Grade A");  
}  
else if (marks >= 75) {  
    System.out.println("Grade B");  
}  
else if (marks >= 60) {  
    System.out.println("Grade C");  
}  
else if (marks >= 50) {  
    System.out.println("Grade D");  
}  
else {  
    System.out.println("Fail");  
}
```

Since marks is 82, the output is:

Grade B

Important Point

The conditions are checked from top to bottom.

Once a condition becomes true, its block is executed and the remaining else-if conditions are skipped.

9. Method

A method is a block of code that performs a specific task.

Instead of writing the same logic repeatedly, we can write it once inside a method and call it whenever required.

Advantages

Code reusability

Easy to understand

Easy to test

Reduces code duplication

Makes programs modular

Basic Syntax

```
returnType methodName(parameters) {  
    // logic  
    return value;  
}
```

Example:

```
static int add(int a, int b) {
```

```
    int result = a + b;  
    return result;  
}  
Here:  
static → method can be called without creating an object  
int → return type  
add → method name  
a, b → parameters  
return result → sends result back to the caller
```

10. Static Method

A static method belongs to the class rather than to an object.

Therefore, a static method can be called directly from another static method such as main().

Syntax

```
static returnType methodName(parameters) {  
    // logic  
    return value;  
}
```

Example

```
public class Demo {
```

```
static int add(int a, int b) {
```

```
    int sum = a + b;
```

```
    return sum;
```

```
}
```

```
public static void main(String[] args) {
```

```
    int x = 10;  
    int y = 20;
```

```
    int result = add(x, y);
```

```
    System.out.println("Result = " + result);
```

```
}
```

```
}
```

Execution

main()

|

| x = 10, y = 20

|

↓

add(10, 20)

|

| sum = 30

|

↓

return 30

|

↓

main()

|

↓

Display 30

11. Method with Parameters and Return Value

A method can receive values from main() through parameters and send the result back using return.

Example:

```
public class Demo {
```



```
public static void main(String[] args) {
```

```
    int price = 1000;  
    int discount = 10;
```

```
    double finalPrice = calculatePrice(price, discount);
```

```
    System.out.println("Final Price = " + finalPrice);
```

```
}
```

```
static double calculatePrice(int price, int discount) {
```

```
    double discountAmount = price * discount / 100.0;
```

```
    double finalPrice = price - discountAmount;
```

```
    return finalPrice;
```

```
}
```

```
}
```

Flow

Read data in main()

↓

Pass values as parameters

↓

Static method

↓

Perform logic

↓

Return result

↓

main()

↓

Display result

Key Rule for Your Assignments

Students should follow this structure:

```
public static void main(String[] args) {
```

```
// 1. Read input
```

```
// 2. Call static method
```

```
// 3. Store returned result
```

```
// 4. Display result
```

```
}
```

```
static returnType methodName(parameters) {
```

```
// Logic
```

```
return result;
```

```
}
```

Input → main() → parameters → static method → logic → return → main() → output

12. Non-Static Method

A non-static method is a method that belongs to an object of a class.

To call a non-static method, we need to create an object of the class.

Syntax

```
returnType methodName(parameters) {
```

```
// logic
```

```
return value;
```

```
}
```

Notice that there is no static keyword.

Example

```
public class Demo {
```

```
public static void main(String[] args) {
```

```
    int x = 10;  
    int y = 20;
```

```
Demo obj = new Demo();
```

```
    int result = obj.add(x, y);
```

```
System.out.println("Result = " + result);
```

```
}
```

```
    int add(int a, int b) {
```

```
        int sum = a + b;
```

```
return sum;
```

}

}

How it works

main()

|

| Create object

↓

Demo obj = new Demo();

|

| Call method

↓

obj.add(10, 20)

|

↓

add() performs logic

|

↓

return 30

|

↓

main()

|

↓

Display result

13. Static Method vs Non-Static Method

Static Method

```
static int add(int a, int b) {
```

```
    return a + b;
```

```
}
```

Call:

```
int result = add(10, 20);
```

Non-Static Method

```
int add(int a, int b) {
```

```
    return a + b;
```

```
}
```

Call:

```
Demo obj = new Demo();
```

```
int result = obj.add(10, 20);
```

15. Non-Static Method with Parameters and Return Value

Example:

```
public class Student {
```

```
    int calculateTotal(int m1, int m2, int m3) {
```

```
        int total = m1 + m2 + m3;
```

```
        return total;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        int mark1 = 80;
```

```
        int mark2 = 75;
```

```
        int mark3 = 90;
```

```
        Student s = new Student();
```

```
        int total = s.calculateTotal(mark1, mark2, mark3);
```

```
        System.out.println("Total = " + total);
```

```
    }
```

```
}
```

Output:

Total = 245

Here:

mark1, mark2, mark3 → values read in main()

s → object

calculateTotal() → non-static method

Marks are passed as parameters

Method performs the logic

return total sends the result back to main()

Data Type | Size | Example

byte | 1 byte | byte age = 25;

short | 2 bytes | short salary = 30000;

```

int | 4 bytes | int marks = 85;
long | 8 bytes | long population = 1400000000L;
float | 4 bytes | float price = 99.50f;
double | 8 bytes | double salary = 45000.75;
char | 2 bytes | char grade = 'A';
boolean | JVM-dependent | boolean status = true;
Operator | Meaning | Example
+ | Addition | a + b
- | Subtraction | a - b
* | Multiplication | a * b
/ | Division | a / b
% | Modulus/Remainder | a % b
Operator | Meaning
> | Greater than
< | Less than
>= | Greater than or equal to
<= | Less than or equal to
== | Equal to
!= | Not equal to
Operator | Example | Equivalent
= | a = 10 | a = 10
+= | a += 5 | a = a + 5
-= | a -= 5 | a = a - 5
*= | a *= 5 | a = a * 5
/= | a /= 5 | a = a / 5
%= | a %= 5 | a = a % 5
Static Method | Non-Static Method
Uses static keyword | Does not use static
Belongs to the class | Belongs to an object
Can be called directly using class context | Requires an object
add(10,20) from main() | obj.add(10,20)
No object required | Object required
main() is static | Normal methods are generally non-static

```